# Best Coding Practices (Revisited)

DATA201/422 Data Wrangling

John Bryant

2026-09-21

# Plan for lectures

- Monday: Useful ideas, techniques
  - Folder and file structure
  - Interfaces
  - Self-documenting code
  - Sanity checking your code
- Tuesday: Human-in-the-loop AI coding demonstration
  - Overview
  - Defining aims, principles for project
  - Iterating

# Structuring folder and files

# Note: We are assuming static inputs

- Static/offline/batch inputs
  - E.g. files downloaded from internet
  - E.g. snapshot from database
- Dynamic/online/streaming inputs
  - E.g. online trading data
  - E.g. live social media posts
- Why assume the static case?
  - Easier
  - More common
  - Many of the principles are the same

# Principles for organising folders, files

- Clean separation of parts
  - Data
  - Code
  - Output
- Do *not* edit Data
  - Easier to audit if you retain the untouched original and do all editing through scripts
  - Also easier to extend or modify project
- Data and Code vs Output
  - Data + Code -> Output
  - Data and Code are persistent; Output is temporary
  - Data and Code are committed to git and Output is not (with a few exceptions - what are they?)

# My default project structure

```
name-of-project/
├── Makefile
├── data/
│   ├── some_data.xlsx
│   └── some_more_data.csv
├── src/
│   ├── an_r_file.R
│   └── an_sql_file.sql
└── out/
    ├── a_figure.pdf
    └── some_outputs.rds
```

# Example

- multi-covid-model
  - data
    - data_fit.zip
    - deaths_observed.zip
    - newdata_nonscaled.zip
    - newdata_scaled.zip
  - Makefile
  - out
    - excess.rds
    - fig_pscore_yearall_age65.pdf
    - fig_pscore_yearall_ageall.pdf
    - fig_pscore_yearlast_age65.pdf
    - fig_pscore_yearlast_ageall.pdf
    - forecast.rds
    - hyper.csv
    - mod.rds
    - pscore_age65.rds
    - pscore_ageall.rds
    - vals_pscore_age65.rds
    - vals_pscore_ageall.rds
  - renv
  - renv.lock
  - src
    - excess.R
    - fig_pscore_yearall.R
    - fig_pscore_yearlast.R
    - forecast.R
    - hyper.R
    - mod.R
    - pscore.R
    - vals_pscore.R

# Directories and file paths

- Current working directory (or just “working directory”)
  - Where the application (e.g. R or Python) is currently looking
  - Python: `os.getcwd()`
  - R: `getwd()`
- Absolute file path
  - Complete description of a file’s location on a computer
  - E.g. `C:\Users\ada\projects\app\main.py` (Windows) or `/Users/ada/projects/app/main.py` (macOS, Linux)
  - The computer knows it’s absolute because of the drive and root (`C:\`) or the initial `/`
- Relative file path
  - Description relative to the current working directory
  - E.g. `app\main.py` or `app/main.py`
  - Internally, the computer turns a relative path into an absolute path by combining the current working directory with the relative file path

# File paths in a project

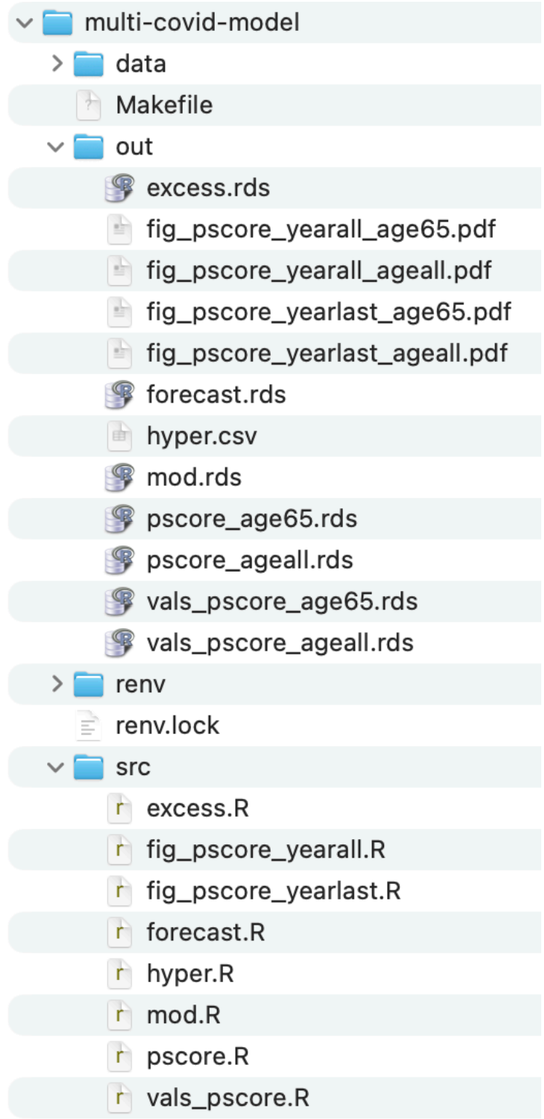
- **Rule 1**: Set the working directory to the “project root” at all times
  - Start the session by opening the project folder
  - Never use `setwd()` or `os.chdir()` in your code
- **Rule 2**: Always use relative file paths (starting from the project root)
  - Never use absolute file paths
  - `"data/dataset1.csv"`, not `"/Users/john/Documents/myproject/data/dataset.csv"`

> *If the first line of your R script is*
>
> ```
> setwd("C:\Users\jenny\path\that\only\I\have")
> ```
>
> *I will come into your office and SET YOUR COMPUTER ON FIRE 🔥.*

Jenny Bryan, https://tidyverse.org/blog/2017/12/workflow-vs-script/

# Lots of small files, not one big file

- Lots of small files, each with a clearly defined task
- My personal preference: one file, one output
- Why are small files better than big ones?
  - Easier to understand (small, digestible pieces)
  - Flexible, reusable - recombine in different ways
  - Easier to fix if something goes wrong



# Clear interfaces (clear inputs, outputs)

# Draw attention to parameters

- A parameter is something that controls behavior
  - E.g. filename, number of records, finishing date, maximum age
- Make parameters visible
  - At the top of the file
  - Sensible names
  - No “magic numbers”

Bad:

```
1 filter(date >= "2020-01-01")
```

Good:

```
1 date_start_study <-  "2020-01-01"
```

…

```
1 filter(date >= date_start_study)
```

# File headers

- Define parameters, read in inputs, load libraries/packages
- Similar to defining the ‘signature’ (the arguments and output value) of a function
- Often helpful to give a 1-2 line description of what the file does

```
## Combine HMD and STMF deaths data to create the 'deaths' file

year_min <- 1970
year_max <- 2025
deaths_path <- "out/deaths.rds"

library(dplyr)
library(purrr)
library(readr)

hmd_deaths <- readRDS("out/hmd/deaths.rds")
stmf_deaths <- readRDS("out/stmf/deaths.rds")
```

# Self-documenting code

# Is code better than comments?

*Why are comments so harmful? They lie! Even with the best of intent, they might not match the actual implementation. They often go stale over time. You can write whatever you want in a comment and there's no consistency checking with the code.*

*You know what doesn't lie? Code. Code doesn't lie. So what do professional programmers do? They don't trust comments and read the code instead.*

Blog post by Bob Carpenter https://statmodeling.stat.columbia.edu/2024/02/07/when-all-else-fails-add-a-code-comment/

(Note that in replies to commenters, Bob Carpenter concedes that comments are useful for some things, e.g. explaining overall aims of code or explaining unexpected choices.)

# Self-documenting code

- Put information into the code itself, not comments
- Applies mainly to production code

# Example: Good choice of names

Bad:

```
1 x <- deaths / population
```

Good:

```
1 death_rate <- deaths / population
```

Bad:

`myplot.png`

Good:

`expected_expenditure_plot.png`

# Example: If you have detailed expectations, record them in the code

Bad:

```
SELECT *
```

Good:

```
SELECT household, income
```

OK:

```
read_xlsx("data.xlsx")
```

Better:

```
read_xlsx(
  path = "data.xlsx",
  col_types = c("numeric", "numeric", "logical", "date")
)
```

# Example: Assertions as documentation

Instead of the comment

```
## Input data must not have duplicates
```

create the assertion

```
if len(df) != len(df.drop_duplicates()):
    raise ValueError("Input data frame contains duplicated rows")
```

Instead of the comment

```
## Deaths must not have NAs
```

create the assertion

```
if (anyNA(deaths))
  stop("'deaths' has NAs")
```

- R and Python have packages for setting up assertions, eg `assertr` and `Pandera`

# Sanity checking your code

# Calculate and compare

- Check one or more rows
  - Calculate the expected result by hand, for one or more rows
  - Compare with the computer output
- Look for weird or missing values in the *last* few rows
  - Function `tail()` in R; method `df.tail()` with pandas in Python
- Look for weird or missing values in some randomly chosen rows

# Graphs

- Best things to graph when data wrangling
  - Input data
  - Final outputs from data wrangling
  - Any intermediate outputs that might be tricky
- Graphs should be *detailed*
  - Can be 50 or 100 pages!
  - Interactive graphs and Shiny apps also good